

AI Presentation Generator Agent — Complete Workflow

A start-to-finish breakdown of the frontend, FastAPI server, LangGraph pipeline, specialized agents, review gates, asset generation, and PowerPoint export.

1. The Starting Point: The Frontend

File: frontend/src/components/PromptForm.jsx

Action: User enters a topic such as “AI in Healthcare”, adds rough notes, selects a visual theme, and clicks Generate.

Flow: React packages the information as JSON and sends an HTTP POST request to the backend.

2. The API Server Receives the Order

File: server.py

Action: FastAPI receives the request at `/api/generate`.

Flow: It creates a unique `task_id`, starts the AI pipeline in a background thread, and immediately returns the `task_id`. React then connects to `/ws/status/{task_id}` for live updates.

3. The Conveyor Belt Starts: LangGraph

Files: core/state.py and core/orchestrator.py

Action: `run_presentation_pipeline()` creates a blank **DeckState**.

Flow: DeckState is the shared box traveling through the workflow. It starts with the user's prompt and is gradually filled with slides, assets, design information, and HTML.

4. Agent 1: The Content Agent

File: agents/content_agent.py

Action: Receives DeckState and drafts around 5–10 slide outlines.

Flow: The LLM creates titles, keeps slides under five bullets, and decides whether each slide needs a diagram or image. The result is stored in DeckState.

5. Gatekeeper 1: Content Review

File: core/orchestrator.py — `content_review_gate`

Action: An LLM acts as the Lead Editor.

Flow: It checks for problems such as marketing fluff or too many bullets. Failed content goes back to the Content Agent; approved content continues.

6. Agents 2 & 3: Asset Generators

Files: agents/diagram_agent.py and agents/image_agent.py

Diagrams: The Diagram Agent creates Mermaid.js code and `tools/mermaid_tool.py` turns it into PNG files.

Images: The Image Agent creates optimized search queries and `tools/stock_photo_tool.py` downloads images from Pexels/Unsplash.

Result: Asset paths are attached to DeckState.

7. Agent 4: The Design Agent

File: agents/design_agent.py

Action: Receives text, diagrams, and images.

Flow: It designs a 16:9 slide, creates HTML/CSS previews, and generates a structured **elements** JSON array containing precise coordinates such as left, top, and width.

8. Agent 5: The Refinement Agent

File: agents/refinement_agent.py

Action: Performs quality assurance.

Flow: It repeatedly checks HTML/CSS and coordinates. If text overflows or alignment is wrong, it fixes the coordinates. Playwright in tools/browser_tool.py then captures screenshots for the frontend gallery.

9. Gatekeeper 2: Design Review

File: core/orchestrator.py — design_review_gate

Action: The LLM acts as Lead Design Director.

Flow: It performs the final audit. If the design is not executive-ready, it routes back to Refinement. If approved, it moves to export.

10. Agent 6: The Export Agent

Files: agents/export_agent.py and tools/pptx_builder_tool.py

Action: Creates the final PowerPoint.

Flow: python-pptx reads the finalized coordinates and creates native, editable PowerPoint text boxes and image shapes. The file is saved as **presentation_deck.pptx**.

11. The Finish Line: The Frontend

Files: server.py and frontend/src/components/DownloadCard.jsx

Action: server.py sends **pipeline_complete** through WebSocket.

Flow: React shows success. The Download button calls **GET /api/download** and delivers the final PowerPoint to the browser.

Complete File Handoff

Stage	Component
1	React frontend
2	server.py
3	orchestrator.py
4	content_agent.py
5	diagram_agent.py + image_agent.py
6	design_agent.py
7	refinement_agent.py
8	export_agent.py
9	server.py
10	React frontend / DownloadCard.jsx

Summary: React collects the request → FastAPI starts the task → LangGraph moves DeckState through specialized agents and review gates → the Export Agent creates the editable PowerPoint → React delivers it to the user.